

Com 212

[Handwritten signature]
226

INTRODUCTION TO SYSTEM PROGRAMMING

System programming involves designing and writing computer programs that allow the computer hardware to interface with the **programmer** and the user, leading to the effective execution of application software on the computer system.

Systems programming involves the development of the individual pieces of software that allow the entire system to function as a single unit. **Systems programming** involves many layers such as the operating system (OS), firmware, and the development environment.

Application Programming aims at producing software which provides services to the user (e.g. word processor, Spreadsheets, databases, Accounting packages).

Systems Programming aims at producing software which provides services to the computer hardware (e.g. disk defragmenter, Operating Systems...).

It also requires a greater degree of hardware awareness that is to say it is machine dependent and the programmer needs to know the hardware within which the software will operate.

DEFINITION OF ASSEMBLER Computer science is not as precise a field as mathematics, so most definitions are not rigorous. An attempt to define Assembler can be formulated as follows: an assembler is a translator that translates source instructions (in symbolic language) into target instructions (in machine language), on a one-to-one basis. This means that each source instruction is translated into exactly one target instruction.

Another good definition of an assembler is a translator that translates a machine-oriented language into machine language.

TYPES OF ASSEMBLERS AND LOADERS

- 1) **A One-pass Assembler:** One that performs all its functions by reading the source file once.
- 2) **A Two-Pass Assembler:** One that reads the source file twice.
- 3) **A Resident Assembler:** One that is permanently loaded in memory.
- 4) **A Macro-Assembler:** One that supports macros.
- 5) **A Cross-Assembler:** An assembler that runs on one computer and assembles programs for another.
- 6) **A Meta-Assembler:** One that can handle many different instruction sets.
- 7) **A Pre-assembler:** This, in a sense, is the opposite of an assembler. It translates machine code into a source program in assembler language.

AN OPERATING SYSTEM (commonly abbreviated computer system that is responsible for the management and coordination of activities and the sharing of the limited resources of the computer. The operating system acts as a host for applications that are run on the machine. As a host, one of the purposes of an operating system is to handle the details of the operation of the

hardware. This relieves application programs from having to manage these details and makes it easier to write applications. Almost all computers, including handheld computers, desktop computers, supercomputers, and even video game consoles, use an operating system of some type.

THE ONE-PASS ASSEMBLER as its name implies, this assembler reads the source file once. During that single pass, the assembler handles both label definitions and assembly. The only problem is future symbols.

Work of one-pass

It generates instructions by evaluating the mnemonics (symbols) in operation field and find the value of symbol and literals to produce machine code.

Advantages:

- it is necessary or desirable to avoid a second pass over the source program
- the external storage for the intermediate file between two passes is slow or is inconvenient to use

Disadvantages:

- Main problem: Forward references to both data and instructions
- One simple way to eliminate this problem: require that all areas be defined before they are referenced.

TWO 2-PASS ASSEMBLER A two-pass assembler is easier to understand. Such an assembler performs two passes over the source file. In the first pass it reads the entire source file, looking only for label definitions. All labels are collected, assigned values, and placed in the symbol table in this pass. No instructions are assembled and, at the end of the pass, the symbol table should contain all the labels defined in the program. In the second pass, the instructions are again read and are assembled, using the symbol table.

Advantages

It detects redefinitions

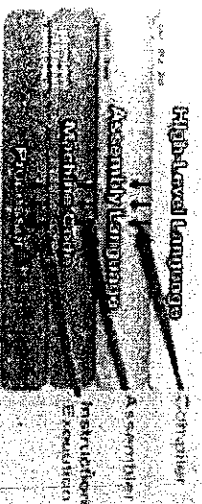
Disadvantages

It is a slower process as 2 passes take more time than 1 pass.

UNDERSTAND THE COMPILATION PROCESS.

Translators, compilers, interpreters and assemblers are all software programming tools that convert code into another type of code, but each term has specific meaning. All of the above work in some way towards getting a high-level programming language translated into machine code that the central processing unit (CPU) can understand. Examples of CPUs include those made by Intel (e.g., x86), AMD (e.g., Athlon APU), NXP (e.g., PowerPC), and many others. It's important to note that all translators, compilers, interpreters and assemblers are programs themselves.

Code Translation



Translators

The most general term for a software code converting tool is "translator." A translator, in software programming terms, is a generic term that could refer to a compiler, assembler, or interpreter; anything that converts higher-level code into another high-level code (e.g., Basic, C++, Fortran, Java) or lower-level (i.e., a language that the processor can understand), such as assembly language or machine code. If you don't know what the tool actually does other than that it accomplishes some level of code conversion to a specific target language, then you can safely call it a translator.

Compilers

Compilers convert high-level language code to machine (object) code in one session. Compilers can take a while, because they have to translate high-level code to lower-level machine language all at once and then save the executable object code to memory. A compiler creates machine code that runs on a processor with a specific Instruction Set Architecture (ISA), which is processor-dependent. For example, you cannot compile code for an x86 and run it on a MIPS architecture without a special compiler. Compilers are also platform-dependent. That is, a compiler can convert C++ for example, to machine code that's targeted at a platform that is running the Linux OS. A cross-compiler, however, can generate code for a platform other than the one it runs on itself.

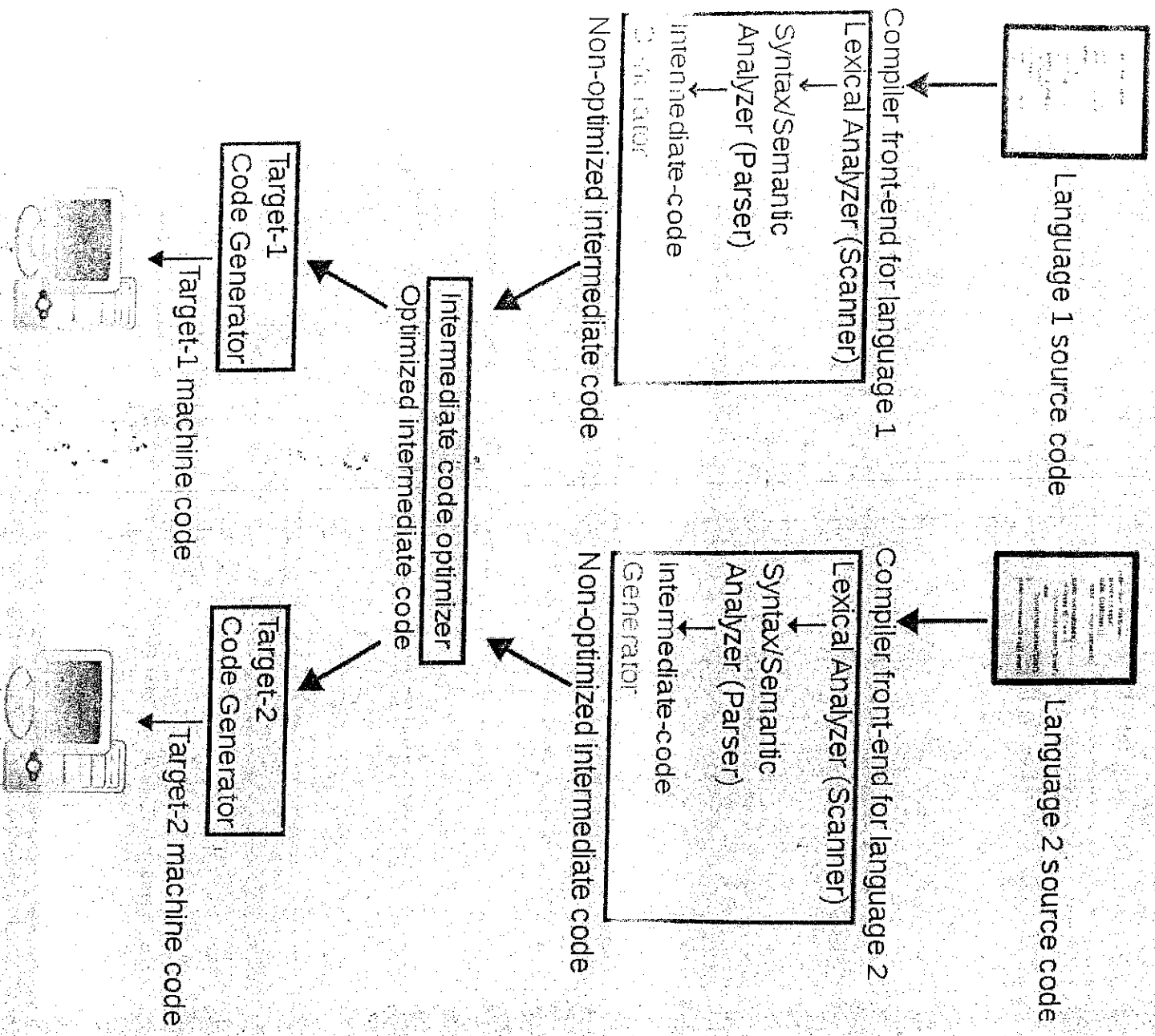
A cross-compiler running on a Windows machine, for instance, could generate code that runs on a specific Windows operating system or a Linux (operating system) platform. Source-to-source compilers translate one program, or code, to another of a different language (e.g., from Java to C). Choosing a compiler then, means that first you need to know the ISA, operating system, and the programming language that you plan to use. Compilers often come as a package with other tools, and each processor manufacturer will have at least one compiler or a package of software development tools (that includes a compiler). Often the software tools (including compiler) are free; after all, a CPU is completely useless without software to run on it. Compilers will report errors after compiling has finished.

Interpreters

Another way to get code to run on your processor is to use an interpreter, which is not the same as a compiler. An interpreter translates code like a compiler but reads the code and immediately executes on that code, and therefore is initially faster than a compiler. Thus, interpreters are often used in software development tools as debugging tools, as they can execute a single line of code at a time. Compilers translate code all at once and the processor then executes upon the machine language that the compiler produced. If changes are made to the code after compilation, the changed code will need to be compiled and added to the compiled code (or perhaps the entire program will need to be re-compiled.) But an interpreter, although skipping the step of compilation of the entire program to start, is much slower to execute than the same program that's been completely compiled.

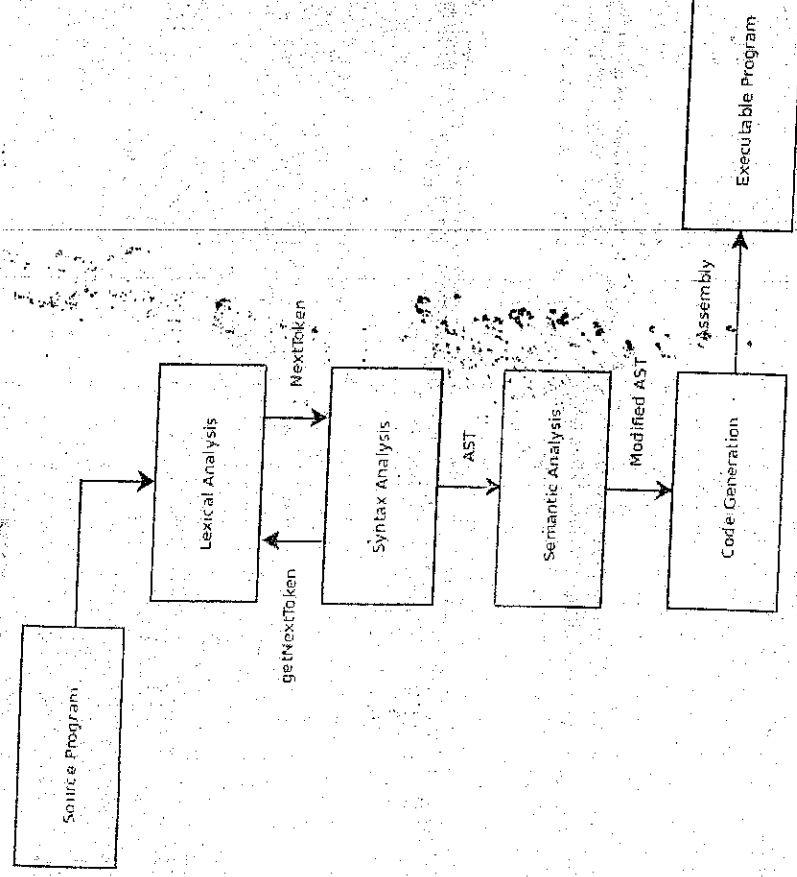
What is a Single Pass Compiler?

There are a number of stages in the compilation process. A single pass compiler makes a single pass through the source text, parsing, analyzing, and generating code only once. In other words, it allows the source code to pass through each compilation unit only once. It immediately translates each code section into its final machine code.



Main stages of single pass compiler are lexical analysis, syntactical analysis and code generator. First, the lexical analysis scans the source code and divides it into tokens. Every programming language has a grammar. It represents the syntax and legal statements of the language. Then, the syntactical analysis determines the language constructs described by the grammar. Finally, the code generator generates the target code. Overall, single pass compiler does not optimize the code. Moreover, there is no intermediate code generation

A **Multipass Compiler** makes the source code go through parsing, analyzing, generating, etc. multiple times while generating intermediate code after each stage. It converts the program into one or more intermediate representations in steps between source code and machine code. It reprocesses the entire compilation unit in each sequential pass. Each pass takes the result of the previous pass as the input and creates an intermediate output. Likewise, in each pass, the code improves until the final pass generates the final code. A multipass compiler performs additional tasks such as intermediate code generation, machine dependent code optimization and machine independent code optimization.



There are multiple advantages to multipass compilers. They are machine independent and include a modular structure. The code generation decouples from the other steps of compilation. Therefore, the passes can be reused for different machines or hardware. Furthermore, it represents more complex and expressive languages

SINGLE PASS COMPILER VERSUS MULTIPASS COMPILER

SINGLE PASS COMPILER	MULTIPASS COMPILER
- Requires compilation unit only once, immediately translating each code section into its final machine code - Faster than multipass compiler - Called a narrow compiler - Has a limited scope - There is no code optimization - There is no intermediate code generation - Takes a minimum time to compile - Memory consumption is lower - Used to implement programming languages such as Pascal	- or abstract syntax tree of a program several times - Slower as each pass reads and writes an intermediate file - Called a wide compiler - Has a great scope - There is code optimization - There is intermediate code generation - Takes some time to compile - Memory consumption is higher - Used to implement programming languages such as Java

Key Difference b/t Compiler & Interpreter.

- Compiler transforms code written in a high-level programming language into the machine code, at once, before program runs, whereas an Interpreter convert each high-level program statement, one by one, into the machine code, during program run.
- Compiled code runs faster while interpreted code runs slower.
- Compiler displays all errors after compilation, on the other hand, the Interpreter displays errors of each line one by one.
- Compiler is based on translation linking-loading model, whereas Interpreter is based on Interpretation Method.
- Compiler takes an entire program whereas the Interpreter takes a single line of code.

Tokens and Delimiters

Often an input String contains several groups of characters, where each group acts as a unit that has significance. Such a group is called a **token**. Tokens are separated from each other by delimiter characters. For example, say that you need to add up the following numbers:

You automatically group these characters to form four integers separated by spaces. The *tokens* in this case are 12 and 8 and 5 and 32. These tokens are delimited by the space character. What makes a token and what makes a delimiter depends on the application.

Error handling

refers to the routines in a program that respond to abnormal input or conditions. The quality of such routines is based on the clarity of the error messages and the options given to users for resolving the problem. Contrast with "exception handling," which deals with responses to abnormal conditions that are built into the programming language or the hardware

UNDERSTAND THE USE OF UTILITIES AND LIBRARIES.

A program that performs a specific task related to the management of computer functions, resources, or files, as password protection, memory management, virus protection, and file compression.

(computing) Any of a large range of software, often included with the operating system of a mainframe or personal computer, that runs specific tasks associated with the computer or its environment.

Software that plays a supporting role for users and developers. Basic utility programs include file/folder management (copy, move, etc.), file search and compare, disk format and partition, as well as diagnostic routines to check performance and the health of the hardware.

Examples of the utility programs (Utilities) include: Disk defragmenters, System Profilers, Network Managers, Application Launchers, Antivirus software, Backup software, Disk repair, Disk Cleaners, Registry Cleaners, Disk Space analyzer, file manager, File Compression, Data Security and many more.

Functions of Utility Programs

- **System Utilities:** Some of the system utility programs are memory manager, antivirus and firewall, registry checker and cleaner, package installer and explorer. Also, with the help of such system utility programs, users can execute functions that are crucial for smooth running of an operating system.
- **File Management Utilities:** include tools such as data archivers, software backup tools, file compression tools and managers. With the help of these, users can manage their data in the form of files and folders. These utilities help users to sort out, store and categorize files according to the requirement.
- **Storage device management utility:** programs provide solutions for enhancing disk capacity, such as disk clean-up, partition management, formatting, disk space allocation, defragmentation, etc. With the help of this utility program, users can compartmentalise systems and external drives for efficient management of programs and files that are stored within.
- **Miscellaneous Utilities:** Apart from these three utility program categories, there are various other programs that help in managing business operations. Some of these programs include data generators, HTML checkers, and hex editors, to name a few.

Common Tasks Performed by Utility Programs

- Disk Defragmentation
- Disk Clean-up
- File Management
- Compression
- Disk Management
- Antivirus and Firewall

LIBRARY IN COMPUTING, a library is a collection of subroutines or classes used to develop software. Libraries contain code and data that provide services to independent programs. This allows code and data to be shared and changed in a modular fashion. Some executables are both standalone programs and libraries, but most libraries are not executables. Executables and libraries

make references known as *links* to each other through the process known as *linking*, which is typically done by a linker.

A collection of standard routines used in computer programs, usually stored as an executable file.

TYPES OF LIBRARIES

- **Static Libraries:** Historically, libraries could only be *static*. A static library, also known as an *archive*, consists of a set of routines which are copied into a target application by the compiler, linker, or binder, producing object files and a stand-alone executable file. This process, and the stand-alone executable file, are known as a static build of the target application.
- **Dynamic Linking:** Dynamic linking means that the subroutines of a library are loaded into an application program at runtime, rather than being linked in at compile time, and remain as separate files on disk. Only a minimum amount of work is done at compile time by the linker; it only records what library routines the program needs and the *index* names or numbers of the routines in the library.
- **Shared Libraries:** In addition to being loaded statically or dynamically, libraries are also often classified according to how they are shared among programs. Dynamic libraries almost always offer some form of sharing, allowing the same library to be used by multiple programs at the same time. Static libraries, by definition, cannot be shared.
- **Remote Libraries:** Another solution to the library issue is to use completely separate executables (often in some lightweight form) and call them using a remote procedure call (RPC) over a network to another computer.
- **Object Libraries:** Although dynamic linking was originally developed in the 1960s, it did not reach consumer operating systems until the late 1980s; it was generally available in some form in most operating systems by the early 1990s. It was during this same period that object-oriented programming (OOP) was becoming a significant part of the programming landscape.
- **Locating Libraries at Runtime:** Dynamic linkers/loaders vary widely in functionality. Some depend on explicit paths to the libraries being stored in the executable. Any change to the library naming or layout of the file system will cause these systems to fail. More commonly, only the name of the library (and not the path) is stored in the executable, with the operating system supplying a system to find the library on-disk based on some algorithm.

The relationship b/w Utilities & Libraries

Libraries are not technically programs, but, instead reusable program fragments used by applications to reduce redundant code and provide a central API for common functions, such as reading a JPEG or decoding MP3 audio. **Utilities** are programs used to manage the system, such as formatting disk drives or adjust the power performance settings. There is little relation to the two, except that utilities may use libraries to reduce their overall code size.

INTRODUCTION TO OPERATING SYSTEM

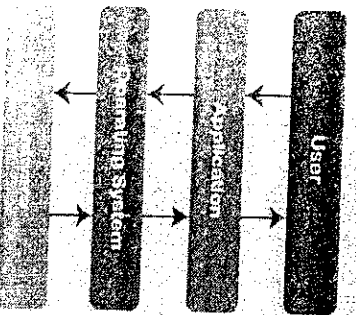
History of OS

- ✓ Operating systems were first developed in the late 1950s to manage tape storage
- ✓ The General Motors Research Lab implemented the first OS in the early 1950s for their IBM 701
- ✓ In the mid-1960s, operating systems started to use disks
- ✓ In the late 1960s, the first version of the Unix OS was developed
- ✓ The first OS built by Microsoft was DOS. It was built in 1981 by purchasing the 86-DOS software from a Seattle company
- ✓ The present-day popular OS Windows first came to existence in 1985 when a GUI was created and paired with MS-DOS.

An **Operating System** acts as a communication bridge (interface) between the user and computer hardware. The purpose of an operating system is to provide a platform on which a user can execute programs in a convenient and efficient manner.

An operating system is a piece of software that manages the allocation of computer hardware. The coordination of the hardware must be appropriate to ensure the correct working of the computer system and to prevent user programs from interfering with the proper working of the system.

Example: Just like a boss gives order to his employee, in the similar way we request or pass our orders to the Operating System. The main goal of the Operating System is to thus make the computer environment more convenient to use and the secondary goal is to use the resources in the most efficient manner.



What is Operating System?

An operating system is a program on which application programs are executed and acts as a communication bridge (interface) between the user and the computer hardware.

The main task an operating system carries out is the allocation of resources and services, such as allocation of: memory, devices, processors and information. The operating system also includes programs to manage these resources, such as a traffic controller, a scheduler, memory management module, I/O programs, and a file system. These are the popular operating systems: Linux OS, Windows OS, Mac OS, VNIS, OS/400 etc.

Function of Operating System

1. **Security** – The operating system uses password protection to protect user data and similar other techniques. It also prevents unauthorized access to programs and user data.
2. **Control over system performance** – Monitors overall system health to help improve performance. Records the responsiveness between service requests and system response to have a complete view of the system health. This can help improve performance by providing important information needed to troubleshoot problems.
3. **Job accounting** – Operating system keeps track of time and resources used by various tasks and users. This information can be used to track resource usage for a particular user or group of user.
4. **Error detecting aids** – Operating system constantly monitors the system to detect errors and avoid the malfunctioning of computer system.
5. **Coordination between other software and users** – Operating systems also coordinate and assign interpreters, compilers, assemblers and other software to the various users of the computer systems.
6. **Memory Management** – The operating system manages the Primary Memory or Main Memory. Main memory is made up of a large array of bytes or words where each byte or word is assigned a certain address. Main memory is a fast storage and it can be accessed directly by the CPU. For a program to be executed, it should be first loaded in the main memory. An Operating System performs the following activities for memory management:
 7. **It keeps tracks of primary memory**, i.e., which bytes of memory are used by which user program. The memory addresses that have already been allocated and the memory addresses of the memory that has not yet been used. In multi programming, the OS decides the order in which process are granted access to memory, and for how long. It Allocates the memory to a process when the process requests it and deallocates the memory when the process has terminated or is performing an I/O operation.
 8. **Processor Management** – In a multi programming environment, the OS decides the order in which processes have access to the processor, and how much processing time each process has. This function of OS is called process scheduling. An Operating System performs the following activities for processor management.
 9. **Keeps tracks of the status of processes**. The program which perform this task is known as traffic controller. Allocates the CPU that is processor to a process. De-allocates processor when a process is no more required.
 10. **Device Management** – An OS manages device communication via their respective drivers. It performs the following activities for device management. Keeps tracks of all devices connected to system. Designates a program responsible for every device known as the Input/output controller. Decides which process gets access to a certain device and for how long. Allocates devices in an effective and efficient way. Deallocates devices when they are no longer required.
 11. **File Management** – A file system is organized into directories for efficient or easy navigation and usage. These directories may contain other directories and other files. An Operating System carries out the following file management activities. It keeps track of where information is stored, user access settings and status of every file and more. These facilities are collectively known as the file system.

Types of Operating System

- Batch Operating System
- Time-Sharing Operating System
- Embedded Operating System
- Multiprogramming Operating System
- Network Operating System
- Distributed Operating System
- Multiprocessing Operating System
- Real-Time Operating System.

Batch Operating System

Some computer processes are very lengthy and time-consuming. To speed the same process, a job with a similar type of needs are batched together and run as a group. The user of a batch operating system never directly interacts with the computer. In this type of OS, every user prepares his or her job on an offline device like a punch card and submit it to the computer operator.

Real time OS

A real time operating system time interval to process and respond to inputs is very small. Examples: Military Software Systems, Space Software Systems are the Real time OS example.

Network Operating System

Network Operating System runs on a server. It provides the capability to serve to manage data, user, groups, security, application, and other networking functions.

Multi-Tasking/Time-sharing Operating systems

Time-sharing operating system enables people located at a different terminal(shell) to use a single computer system at the same time. The processor time (CPU) which is shared among multiple users is termed as time sharing.

Features of Operating System (OS)

Here is a list important features of OS:

- Protected and supervisor mode
- Allows disk access and file systems Device drivers Networking Security
- Program Execution
- Memory management Virtual Memory Multitasking
- Handling I/O operations
- Manipulation of the file system
- Error Detection and handling
- Resource allocation
- Information and Resource Protection

Uses of Operating System

The operating system is used everywhere today, such as banks, schools, hospitals, companies, mobiles, etc. No device can operate without an operating system because it controls all the user's commands.

- **linux/unix** operating system is used in the bank because it is a very secure operating system.

- **Symbian OS, Windows Mobile, iOS, and Android OS** are used in mobile phone operating systems as these operating systems are a lightweight operating system.

Advantages of Operating System

- It is helpful to monitor and regulate resources.

- It can easily operate since it has a basic graphical user interface to communicate with your device.
- It is used to create interaction between the users and the computer application or hardware.
- The performance of the computer system is based on the CPU.
- The response time and throughput time of any process or program are fast.
- It can share different resources like fax, printer, etc.
- It also offers a forum for various types of applications like system and web application.

Disadvantage of the Operating System

- It allows only a few tasks that can run at the same time.
- If any error occurred in the operating system, the stored data can be destroyed.
- It is a very difficult task or works for the OS to provide entire security from the viruses because any threat or virus can occur at any time in a system.
- An unknown user can easily use any system without the permission of the original user.
- The cost of operating system costs is very high.

BATCH PROCESSING BATCH Processing can be defined as executing a series of non-interactive jobs all at one time. The term originated in the days when users entered programs on punch cards. They would give a batch of these programmed cards to the system operator, who would feed them into the computer. Batch jobs can be stored up during working hours and then executed during the evening or whenever the computer is idle. Batch processing is particularly useful for operations that require the computer or a peripheral device for an extended period of time. Once a batch job begins, it continues until it is done or until an error occurs. Note that batch processing implies that there is no interaction with the user while the program is being executed. An example of batch processing is the way that credit card companies process billing or Power Holding Company processes their bills. The customer does not receive a bill for each separate credit card purchase or meter reading but one monthly bill for all of that month. The bill is created through batch processing, where all of the data are collected and held until the bill is processed a batch at the end of the billing cycle. The opposite of batch processing is transaction processing or interactive processing. In interactive processing, the application responds to commands as soon as you enter them. **TIME SHARING** This involves the CPU allocating individual slices of time to a number of users on the computer system. As the number of users increases the response time for each terminal declines. The speed of the CPU compared to that of the VDU and terminal is so much faster that it gives the user the impression that they are the sole user of the system.

MULTIPROGRAMMING MULTITASKING AND MULTIPROCESSING SYSTEMS

Multiprogramming: In multiprogramming systems, the running task keeps running until it performs an operation that requires waiting for an external event (e.g. reading from a tape) or until the computer's scheduler forcibly swaps the running task out of the CPU. Multiprogramming systems are designed to maximize CPU usage.

Multitasking: In computing, multitasking is a method by which multiple tasks, also known as processes, share common processing resources such as a CPU. In the case of a computer with a single CPU, only one task is said to be running at any point in time, meaning that the CPU is actively executing instructions for that task. Multitasking solves the problem by scheduling which

task may be the one running at any given time, and when another waiting task gets a turn. The act of reassigning a CPU from one task to another one is called a context switch.

Multiprocessing: Multiprocessing is a generic term for the use of two or more central processing units (CPUs) within a single computer system. There are many variations on this basic theme, and the definition of multiprocessing can vary with context, mostly as a function of how CPUs are defined (multiple cores on one die, multiple chips in one package, multiple packages in one system unit, etc.). Multiprocessing sometimes refers to the execution of multiple concurrent software processes in a system as opposed to a single process at any one instant.

Introduction

PCs use interrupt requests to handle various hardware functions. Hardware interrupts were first introduced by the UNIVAC 1103 in 1953. The first incidence of interrupt masking was incorporated by IBM 650 in 1954. It is essential to assign different IRQs to different hardware devices to perform various functions. During program execution, devices like keyboards, a mouse requires services of CPU and generates an interrupt to get the attention from CPU and process the requested service. These are known as interrupts. One of the buses of the I/O device is dedicated to serving this purpose is known as Interrupt Service Routine (ISR). These are used in various applications such as for time-sensitive events, data transfer, emphasizing on abnormal events, watchdog timers, traps, etc.

What is an Interrupt?

Definition: It is referred to as an input signal that has the highest priority for hardware or software events that requires immediate processing of an event. During the early days of computing, the processor had to wait for the signal to process any events. The processor should check every hardware and software program to understand if there is any signal to be processed. This method would consume a number of clock cycles and makes the processor busy. Just in case, if any signal was generated, the processor would again take some time to process the event, leading to poor system performance.

A new mechanism was introduced to overcome this complicated process. In this mechanism, hardware or software will send the signal to a processor, rather than a processor checking for any signal from hardware or software. The signal alerts the processor with the highest priority and suspends the current activities by saving its present state and function, and processes the interrupt immediately. This is known as ISR. As it doesn't last long, the processor restarts normal activities as soon as it is processed.

*If you interrupt someone who is speaking, you say or do something that causes them to stop.
Turkin tapped him on the shoulder. Sorry to interrupt, Colonel! [VERB]*
He tried to speak, but she interrupted him. [VERB noun]

Types of Interrupt

These are classified into two main types.

Hardware Interrupts

An electronic signal sent from an external device or hardware to communicate with the processor indicating that it requires immediate attention. For example, strokes from a keyboard or an action

from a mouse invoke hardware interrupts causing the CPU to read and process it. So it arrives asynchronously and during any point of time while executing an instruction. Hardware interrupts are classified into two types.

Maskable Interrupts

Processors have to interrupt mask register that allows enabling and disabling of hardware interrupts. Every signal has a bit placed in the mask register. If this bit is set, an interrupt is enabled & disabled when a bit is not set, or vice versa. Signals that interrupt the processors through these masks are referred to as masked interrupts.

Non-maskable Interrupts (NMI) – The NMIs are the highest priority activities that need to be processed immediately and under any situation, such as a timeout signal generated from a watchdog timer.

Software Interrupts

The processor itself requests a software interrupt after executing certain instructions or if particular conditions are met. These can be a specific instruction that triggers an interrupt such as subroutine calls and can be triggered unexpectedly because of program execution errors, known as exceptions or traps.

Triggering Methods

Generally, these signals are designed to trigger using either a logic signal level or a signal edge. These methods are of two types.

Level-triggered Interrupt

In this type, the input module invokes an interrupt if the service level of this is asserted. If an interrupt source continues to be asserted when the firmware interrupt handler handles it, this module regenerates and triggers the handler to invoke again. The level-triggered inputs are not good if remains asserted for a longer duration.

Edge-triggered Interrupt

An edge-triggered interrupt input module invokes an interrupt as soon as it identifies an asserting edge – a falling or a rising edge. The edge becomes noticed when the level of source changes. This type of triggering needs immediate action, irrespective of the activity of the source.

System Implementation

Interrupts shall be applied to hardware as a different component along with control lines, or integrated into memory subsystems. When implementation is performed in hardware, it needs a Programmable Interrupt Controller (PIC) to connect between the CPU's input pin and the interrupting device. PCI multiplexes various sources of interrupt to the single or double CPU line. When implemented pertaining to the memory controller, the system's memory address slot is mapped directly with the interrupts.

Shared Interrupt Requests (IRQs)

With edge-triggered interrupt, a pull-up or pull-down resistor is used to drive the interrupt line. This line transmits every pulse produced by each device. If interrupt pulses generated from different devices happen close in time, the CPU must invoke a trailing edge of the pulse to not to miss interrupts, following which CPU ensures to inspect every device for service requests. A well-behaved Industry Standard Architecture (ISA) motherboards with pull-up resistors having sharing IRQ lines must work fine. However, multiple devices sharing IRQ line in older systems having poorly designed programming interface makes it difficult to process interrupts. On the other hand, a new system architecture, such as PCI helps in providing considerable relief to this problem.

Hybrid

A hybrid type of system implementation has a combination of both edge-triggered and level-triggered signaling. The hardware shall look for an edge and also verifies that if a signal is active for a certain period. A hybrid type is commonly used for the non-maskable interrupt (NMI) input, which ensures that false interrupts do not affect the system.

Message-Signalled

A message-signaled interrupt request for service using device signals by transmitting a message over a communication channel, such as a computer bus. These do not use a physical interrupt line. PCI Express acts as a serial bus and is used exclusively as message-signaled interrupts.

Doorbell

The doorbell interrupt is commonly used as a mechanism by a software system to signal computer hardware to complete the work. Upon mutual agreement between hardware and software, the software places the data in the well-known memory location and rings the doorbell to notify the hardware that the data is ready and waiting to process. Now, the computer hardware device understands the data is valid and processes it accordingly.

Multiprocessor IPI

The multiprocessor systems, an interrupt request from a processor is sent to a different processor through inter-processor interrupts (IPI)

Typical Uses/Applications

These are powerful inputs typically used to

Service hardware timers, handle keyboard strokes and mouse actions

Respond quickly to time-sensitive or real-time events

Data transfer to and from peripheral devices

Responds to high-priority tasks such as power-down signals, traps, and watchdog timers
Indicates abnormal events of CPU

Power-off interrupt forces the loss of power, enabling orderly shut-down of the system
Periodic interrupts to keep track of absolute time

FAQs

1). Why interrupts are used?

These are used to get the attention of the CPU to perform services requested by either hardware or software.

2). What is NMI?

NMI is a non-maskable interrupt, that cannot be ignored or disabled by the processor

3). What is the function of interrupt acknowledge line?

The processor sends a signal to the devices indicating that it is ready to receive interrupts.

4). Describe hardware interrupt. Give examples

It is generated by an external device or hardware; such as keyboard keys or mouse movement invokes hardware interrupts

5). Describe software interrupt.

It is defined as a special instruction that invokes an interrupt such as subroutine calls. Software interrupts can be triggered unexpectedly because of program execution errors

6). Which interrupt has the highest priority?

Non-maskable edge and level triggered

TRAP has the highest priority

7). Give some uses of interrupt

Respond quickly to time-sensitive or real-time events

Data transfer to and from peripheral devices

Responds to high-priority tasks such as power-down signals, traps, and watchdog timers

Indicates abnormal events of CPU

8). What is a hybrid type of system implementation?

A hybrid type of system implementation has a combination of both edge-triggered and level-triggered signaling. The hardware shall look for an edge and also verifies that if a signal is active for a certain period.

In this article, we have understood the importance of interrupts and how these are used to execute the services requested. We also discussed what are the types, system implementation, and its uses.

Assemblers

An assembler translates a program written in assembly language into machine language and is effectively a compiler for the assembly language, but can also be used interactively like an interpreter. Assembly language is a low-level programming language. Low-level programming languages are less like human language in that they are more difficult to understand at a glance; you have to study assembly code carefully in order to follow the intent of execution and in most cases, assembly code has many more lines of code to represent the same functions being executed as a higher-level language. An assembler converts assembly language code into machine code (also known as object code), an even lower-level language that the processor can directly understand.

Assembly language code is more often used with 8-bit processors and becomes increasingly unwieldy as the processor's instruction set path becomes wider (e.g., 16-bit, 32-bit, and 64-bit). It is not impossible for people to read machine code, the strings of ones and zeros that digital devices (including processors) use to communicate, but it's likely only read by people in cases of computer forensics or brute-force hacking. Assembly language is the next level up from machine code, and is quite useful in extreme cases of debugging code to determine exactly what's going on in a problematic execution, for instance. Sometimes compilers will "optimize" code in unforeseen ways that affect outcomes to the bafflement of the developer or programmer such that it's necessary to carefully follow the step-by-step action of the processor in assembly code, much like a hunter tracking prey or a detective following clues

INTERRUPTS

Interrupt hardware was invented to eliminate the need for explicit calls to a polling procedure from within applications code. Essentially all computers on the market today, from the smallest microcontrollers to the highest performance supercomputers include such hardware. In effect, what the basic interrupt mechanism does is check all of the relevant device status bits just after executing each and every machine instruction, inserting a call to an interrupt handler, analogous to our poll routine whenever some device is ready. So long as no devices need service, this allows the computer to execute instructions at full speed. In general, an *interrupt* can be viewed as a hardware-initiated call to a procedure, the *interrupt handler* or *interrupt service routine*. In effect, the instruction execution loop of the central processor has been made to serve as the main polling loop of our application! Although the abstract description of an interrupt as a hardware initiated procedure call applies to most interrupt hardware, the details vary considerably from machine to machine. The address of the interrupt service routine is frequently stored in a special register or dedicated memory location, called the interrupt vector. On some machines, calls to interrupt service routines parallel procedure calls to the extent that the *return* procedure return can be used to return to the code which was running at the time of the interrupt, while on others, special return-from-interrupt instructions must be used. An assembly language *sub* is a bit of code that masks over the incompatibility between one model of control transfer and another: in this case, allowing the use of an interrupt to call a normal function compiled by a compiler that knew nothing about interrupt service routines. Once an interrupt service routine has been called, it is essential that the hardware request for that service be disabled or withdrawn. If it were not, an infinite (and possibly recursive) loop would result in which, after executing the first instruction of the interrupt service routine, the hardware would force a control transfer to the start of the same interrupt service routine. On some systems, the interrupt is automatically disabled by the actions of the central processor hardware when it responds to the interrupt, while on others, the first instruction of each interrupt service routine must disable the interrupt. In the examples presented here, it will be assumed that the hardware automatically disables interrupts as it calls the interrupt service routine.

Even if the hardware can disable interrupts, the software must also be able to enable or disable them. For example, on return from an interrupt service routine, after the software has done whatever the interrupt requested, the software must re-enable the interrupt as it returns to the code which was interrupted. Furthermore, if the output queue is empty, there is no point in responding to an interrupt from the output device when it is ready to transfer more data; a similar argument can be made when the input queue is full. On most machines, there are special instructions to enable and disable interrupts; on some, these instructions apply to all interrupts at the same time, while other machines allow groups of devices to be enabled or disabled as a group. In addition, it is usual to include, in each device's control register, one or more interrupt enable bits corresponding to each condition the device can sense that might be cause for an interrupt request. If the interrupt enable bit for a condition is set, then when that condition is detected, there will be an interrupt

request. It should be noted that the disabling an output device's ability to request interrupts when the output queue is empty and enabling that device when data is put in the queue is a special purpose solution to a general problem, the producer-consumer problem. The device is the consumer, and the application is the producer, and the general problem is to prevent the consumer from attempting to use data that has not yet been produced. In any producer-consumer system, whether the system is all in hardware, all in software, or mixed between the two, there must be some synchronization mechanism to make the consumer wait until data is available!

Interrupt handlers and the scheduler.

Since an interrupt handler blocks the highest priority task from running, and since real time operating systems are designed to keep thread latency to a minimum, interrupt handlers are typically kept as short as possible. The interrupt handler defers all interaction with the hardware as long as possible; typically, all that is necessary is to acknowledge or disable the interrupt (so that it won't occur again when the interrupt handler returns). The interrupt handler then queues work to be done at a lower priority level, often by unblocking a driver task (through releasing a semaphore or sending a message). The scheduler often provides the ability to unblock a task from interrupt

handler

context.

COURSE CONTENT

- ✓ General concept of system programming
- ✓ Assembler and assembler process
- ✓ The compilation process
- ✓ The use of utilities and libraries.
- ✓ The function of operating system
- ✓ Input/output handlers